

# Prelaunch Checker - Tech Stack

## Contents

|                                                     |   |
|-----------------------------------------------------|---|
| Why TS? . . . . .                                   | 1 |
| Why Nest? . . . . .                                 | 2 |
| Why Go for event ingestion? . . . . .               | 2 |
| Why Postgres? . . . . .                             | 3 |
| Why is ClickHouse for Analytics DB? . . . . .       | 4 |
| Why Redis and BullMQ? . . . . .                     | 4 |
| Why Crawler in Playwright? . . . . .                | 5 |
| Why rrweb (Client SDK)? . . . . .                   | 6 |
| Why AWS Fargate, RDS, ElastiCache and S3? . . . . . | 7 |

### Backend Framework:

NestJS + TS

### Event Ingestion:

Node/Fastify - now

Go - later

### Frontend Framework:

React.js

### DB:

PostgreSQL for relational data

ClickHouse for events and analytics

Redis and BullMQ for caching and queuing

### Crawler:

Playwright

### Session replays:

rrweb (client SDK)

S3

**Infra:** AWS Fargate/RDS/ElastiCache/S3

---

## Why TS?

1. **Shared types across services.** Our apps/api, apps/web, and packages/shared-types can share one SeoAuditReport interface. If I change a field name in the backend, Type-

Script immediately flags every frontend component still using the old name — instead of us finding out when a user reports a blank dashboard.

2. **Refactoring safety.** Renaming `checks.imagesAlt` to `checks.missingAltText` across 40 files is a mechanical, safe operation with TS — the IDE does it for me. In plain JS it's grep and hope.
3. **Self-documenting APIs.** A function signature like `function scoreModule(checks: SeoCheckResult[]): ModuleScore` tells you what goes in and out without opening the implementation. In JS you'd be reading the function body or the docs, if they exist and are current.
4. **Autocomplete.** JS editors guess based on patterns; TS knows the real shape of our data, so autocomplete on `report.checks` actually lists our real check keys.

## Why Nest?

1. Dependency injection — services get injected rather than manually imported/instantiated everywhere, which makes testing (mocking a service) far easier.
2. Built-in validation pipes — class-validator decorators on a DTO (`@IsUrl()` `url: string`) reject malformed requests before they touch our business logic, instead of me writing `if (!url) throw...` by hand in every controller.
3. Guards/interceptors — auth checks, logging, and rate limiting are cross-cutting concerns applied declaratively (`@UseGuards(JwtAuthGuard)`) instead of copy-pasted middleware in every route.
4. First-class WebSocket/microservice support — relevant later if our real-time dashboard (live session recordings, live crawl status) needs sockets; Nest has this built in rather than bolted on.

## Why Go for event ingestion?

### 1. Goroutines make “accept many, batch, flush” cheap and simple.

The ingestion service's core loop is: receive event → hold it in a buffer → every N events or N milliseconds, flush the buffer as one bulk insert. In Go this is a natural fit — a goroutine per connection is extremely lightweight (a few KB of stack, not an OS thread), so handling thousands of concurrent open connections doesn't strain memory the way thread-per-connection models do in other languages. Node handles concurrency too (via its event loop), but it's single-threaded for CPU work — Go can actually use multiple cores in parallel for this without reaching for a clustering setup.

### 2. Predictable performance under load — no garbage collector pauses that show up as visible stutter.

Go's GC is tuned for low-latency, and the language itself has very low overhead per request (compiled binary, no JIT warm-up). At the exact moment the ingestion service is under the most pressure (a viral traffic spike on a customer's site means way more events), we want the flattest possible latency curve, not the occasional GC pause that Node can hit under memory pressure.

### 3. Memory footprint stays small and stable.

An ingestion service that's buffering thousands of small event objects benefits from Go's low per-object overhead. Same logic in Node, at the same volume, tends to use meaningfully more memory per equivalent throughput — not disqualifying, but the kind of thing that turns into a real AWS bill difference at scale.

#### 4. It's a small, self-contained service — so the “cost” of a second language is low.

This is the actual condition that makes it worth it. Rewriting our entire backend in Go would be a real tax — different ecosystem, different idioms, context-switching on every task. But ingestion is maybe 200-400 lines of code with one job: HTTP handler → validate → buffer → batch insert. It's the kind of service we write once, it works, and we rarely touch it again. Paying a one-time learning cost for a service we'll barely revisit is a much better trade than paying that cost across our whole, constantly-changing API.

**Why it can be introduced later?** Because we don't have real load yet to prove it needs Go. If and when we have actual production traffic and profiling shows ingestion is the thing struggling, that's the moment reaching for Go pays for itself — not before.

### Why Postgres?

#### 1. JSON handling — this one matters a lot for us specifically

Our reports store nested, variable-shaped data — checks: { title: {...}, ssl: {...}, brokenLinks: {...} }, SEO audit results with modules and sub-checks, crawl results with a pages array. This is exactly the kind of semi-structured data relational databases weren't originally built for.

MySQL's JSON column stores JSON as a blob you can query into, but indexing into it is limited and clunky (generated columns as a workaround).

Postgres's JSONB type is a proper first-class citizen:

```
-- find every report where the SSL check failed, directly inside the JSON
SELECT * FROM reports WHERE checks->'ssl'->'status' = 'fail';
```

```
-- index directly into a JSON field for fast lookups
CREATE INDEX idx_ssl_status ON reports ((checks->'ssl'->'status'));
```

JSONB is stored in a binary format (not text), so it's faster to query and can be indexed with GIN indexes for arbitrary key lookups. For a product where every report/audit/analytics payload is a nested object, this is the difference between querying it naturally and fighting the database.

#### 2. Extensions — this is a real practical win at our scale

Postgres has an extension ecosystem that adds capabilities we'll likely want:

- **PostGIS** — if we ever do geo-based analytics (visitor locations, regional performance)
- **pg\_cron** — schedule recurring jobs inside the database (an alternative/complement to BullMQ for simple recurring tasks)
- **TimescaleDB** — turns Postgres into a legitimate time-series database; relevant if we ever decide ClickHouse is overkill early on and want to start with Postgres+Timescale for events before graduating
- **pg\_stat\_statements** — built-in query performance monitoring, genuinely useful when debugging slow queries solo with no DBA to ask

MySQL doesn't have an equivalent plugin culture — it's a more closed, “this is what you get” system.

#### 3. Concurrency model — better behaviour under mixed read/write load

Postgres uses MVCC (Multi-Version Concurrency Control) more thoroughly than MySQL's default engine (InnoDB, which also has MVCC but behaves differently under lock contention). Practically: Postgres tends to handle a mix of long-running reads (someone loading a big report) and concurrent writes (new crawl results coming in) with fewer lock conflicts. This matters once we have real concurrent users, not so much on day one.

#### 4. Standards compliance & advanced SQL features

Postgres supports window functions, CTEs (WITH queries), full outer joins, and array types more completely and for longer than MySQL has. If our reporting logic ever needs something like “rank each site’s score against the previous 5 audits” (window functions) or recursive queries (site hierarchy, crawl link graphs), Postgres just handles it more naturally. MySQL has caught up on some of this in recent versions, but Postgres has had it longer and more fully implemented.

#### 5. Data integrity — stricter by default

Postgres enforces constraints (foreign keys, check constraints, data types) more strictly out of the box. MySQL, especially in default/legacy configurations, has historically been more permissive (silently truncating data that doesn’t fit, weaker type enforcement) — modern MySQL with STRICT\_TRANS\_TABLES mode narrows this gap, but Postgres is strict by default without us having to remember to configure it.

#### Why is ClickHouse for Analytics DB?

In Postgres, a row is stored together on disk:

- [event\_id, session\_id, url, x, y, timestamp, device, ...] ← one full row read at a time

To compute “average scroll depth by page,” Postgres still has to read every column of every matching row off disk, even though we only care about two columns (page, scroll\_depth).

ClickHouse stores data column by column:

- event\_id: [1, 2, 3, 4, 5, ...]
- scroll\_depth: [40, 82, 15, 90, 33, ...]
- page: [/home, /pricing, /home, /about, /home, ...]

To answer the same aggregate query, it reads only the page and scroll\_depth columns — nothing else touches disk. On tables with dozens of event fields, this alone is often a 10-100x difference in query speed for exactly the kind of “aggregate over millions of rows” queries a heatmap or dashboard chart needs to run.

#### Why does this matter concretely for our product?

1. **Heatmaps** = aggregate thousands of (x, y) click coordinates per page into a density grid. Pure aggregation query.
2. **Session/funnel analytics** = “how many sessions reached step 3 of the funnel” across millions of events. Pure aggregation.
3. **Core Web Vitals dashboards** = time-series averages/percentiles (p75 LCP over the last 30 days) across huge volumes. This is ClickHouse’s literal bread and butter — it has built-in functions for exactly this (quantile(), time-bucketed aggregations).

#### Why Redis and BullMQ?

**Redis:** Redis’s usefulness here isn’t tied to any particular language or database choice — it solves problems that exist in every version of this stack:

1. **Caching.** Reports, SEO audit results, dashboard aggregates — anything expensive to compute or query (especially against ClickHouse aggregations) benefits from a cache in front of it. Redis is the standard here regardless of whether our DB is MySQL, Postgres, or ClickHouse — nothing about the DB choice changes this need.

**2. Session/rate-limiting.** JWT auth, login rate limiting, API rate limiting per user/site — all typically backed by Redis because it’s fast, has built-in TTL/expiry (a rate-limit counter that auto-resets is trivial), and every framework (Nest, Fastify) has mature libraries for it.

**3. Pub/Sub for real-time features.** If we want live crawl-status updates or live dashboard numbers pushed over WebSockets (the NestJS gateway mentioned earlier), we need something to broadcast “this job just finished” from our worker process to our API process, since they’re separate processes/containers. Redis pub/sub is the standard lightweight way to do that — we don’t need Kafka-level infrastructure for this.

**4. Short-term buffering for the ingestion pipeline.** Even with a Go ingestion service, we likely want a small buffer between “event received” and “batch flushed to ClickHouse” so a service restart doesn’t lose in-flight events. Redis (or Redis Streams specifically) is a lightweight way to do that without introducing Kafka/Redpanda unless we actually need that level of durability guarantee.

**BullMQ:** It’s important to separate two different kinds of “async work” in this system, because they need different tools:

1. High-volume analytics events (clicks, scrolls) → these get batched and written by the ingestion service, not queued as individual BullMQ jobs. Routing thousands of tiny events/second through a job queue would be the wrong tool — too much overhead per item.
2. Discrete, one-off background tasks → this is what BullMQ is actually for, and we still have plenty of these:
  - Running a scheduled site check (daily/weekly re-scan)
  - Generating a PDF report
  - Running a crawl job (Playwright, multi-page)
  - Sending a notification/email when a scheduled check finds a new issue

## Why Crawler in Playwright?

**What actually changed between Puppeteer and Playwright** Both came out of roughly the same lineage (Playwright was built by ex-Puppeteer engineers at Microsoft after they left Google), so they’re similar in API shape — this isn’t a “totally different paradigm” switch. The differences are specific and mostly matter more as our crawler’s job gets more demanding.

**1. Multi-browser support** — Puppeteer is Chromium-only (mostly), Playwright isn’t.

Puppeteer’s whole design is built around the Chrome DevTools Protocol, so it’s fundamentally a Chromium tool (it added limited experimental Firefox support later, but it’s not first-class). Playwright drives Chromium, Firefox, and WebKit (Safari’s engine) through one consistent API.

Why this matters for a crawler specifically:

Some sites render or behave differently across engines — a site might look fine in Chromium but break in Safari/WebKit due to a CSS or JS quirk. Since part of our product’s pitch is “we tell you if your site actually works,” testing only in one engine would be a real blind spot.

**2. Auto-waiting — fewer flaky crawls.**

This is the one that actually saves real engineering time. Puppeteer often requires manually waiting for things:

```
await page.waitForSelector('.price');
await page.click('.price');
```

Playwright's actions auto-wait for the element to be visible, stable (not animating), and receiving events before acting on it:

```
await page.click('.price'); // waits for all of the above automatically
```

For a crawler hitting thousands of arbitrary, unpredictable third-party sites (not pages we control), flaky waits are a real operational problem — a page that's slow to hydrate causes a false “broken” result instead of Playwright correctly waiting it out. This directly reduces false positives in our audit results.

### 3. Better network interception and control.

Playwright's network interception API (`page.route()`) is more ergonomic for the kind of things our crawler does — spoofing an AI-bot user agent to test anti-bot blocking, intercepting and inspecting responses for broken-link detection, blocking image/font loading to speed up crawls when we don't need visuals. Puppeteer can do all this too, but Playwright's API is generally considered cleaner here.

### 4. Built-in test/trace tooling.

Playwright ships with tracing (a full recording of a run — network, console, screenshots, DOM snapshots — that we can replay later) out of the box. When a crawl fails on some random customer site at 3am and we need to debug why without reproducing it live, a trace file is genuinely useful. Puppeteer doesn't have an equivalent built in.

### 5. Actively the more invested-in project right now.

Not a technical point exactly, but a practical one: Playwright currently has more active development momentum and community adoption for new projects. Puppeteer isn't abandoned, but Playwright is where most new tooling in this space (including parts of Screaming Frog's own JS rendering) has been trending.

## Why rrweb (Client SDK)?

rrweb's core idea: instead of capturing what the screen looked like, capture what changed in the DOM — and replay it later by rebuilding those changes in a real browser.

### The naive approach, and why it's wrong

One might think session replay = literally recording video of the user's screen (like a screen recorder). That's how some very early tools worked, and it's a bad approach for a web product:

- Massive file size — video of every session, for every visitor, on every monitored site, is enormous storage and bandwidth cost.
- No structure — we can't query “which sessions clicked this button” from a video file. It's opaque pixels.
- Privacy is hard — we can't selectively mask a credit card field in a video the way we can mask a DOM node.

### What rrweb actually does instead: record the DOM, not the pixels

Concretely:

1. **On page load**, it takes a full serialized snapshot of the DOM (essentially “here's every element and its current state”).
2. **After that**, it listens to the browser's MutationObserver API and records every subsequent DOM change as a small, structured event — a class added, a text node changed, an element inserted/removed, an input value updated, scroll position changed, mouse position changed.
3. **Each of these events is tiny** — a few bytes describing “this node's attribute changed to X” — not a pixel frame.

On replay, rrweb takes that same snapshot plus the stream of DOM-mutation events and re-plays them into a sandboxed iframe, literally rebuilding the page as it changed in real time — clicks, typing, scrolling, animations, all reconstructed from structured data, not decoded from video.

Why this is better than video, concretely

**1. Size.** A DOM-mutation event stream compresses extremely well and is typically a small fraction of the size of equivalent video — because most of a session is static (nothing changing), and rrweb only records deltas. Video has to encode every frame regardless of whether anything moved.

**2. It's structured data,** so it's queryable. Because it's not pixels, rrweb data can be cross-referenced with our click/scroll event analytics — “show me the session replay for the visitor who rage-clicked on the checkout button” is a database query joining rrweb session IDs with our click-event table, not a manual scrub through hours of video.

**3. Selective privacy masking at the DOM level.** Since rrweb sees actual DOM nodes (not pixels), we can tell it “never record the actual text inside any or any field tagged data-clarity-mask” — the masking happens before data ever leaves the browser, not as a blur filter applied after the fact to video. This is exactly how Clarity/Hotjar handle GDPR-sensitive fields, and it's meaningfully more trustworthy than “we recorded everything then blurred it.”

**4. Faithful, resizable replay.** Because we're rebuilding real DOM in a real browser, replay respects the actual page — we can resize the replay viewport, it handles responsive layouts correctly, and it's pixel-perfect to what actually rendered, rather than a lossy video codec's approximation.

Why it's the specific library to use, not build our own

Recording DOM mutations sounds simple but has a lot of nasty edge cases rrweb has already solved: canvas/WebGL content (which MutationObserver can't see, since it's pixels drawn imperatively — rrweb has special plugins to sample canvas frames), iframes, shadow DOM, CSS-in-JS style recalculation, font loading, scroll position on nested scrollable containers, and cross-browser quirks in how mutations fire. This is genuinely gnarly to get right, and it's exactly why Clarity, Hotjar, PostHog, and most other session-replay products either use rrweb directly or something architecturally identical to it — it's a solved problem, not worth re-solving solo.

Why it's a client-side SDK specifically (not server-side)

DOM mutations only exist in the browser — the server never sees the DOM, only whatever data the page sends it. So the recording has to happen client-side, as a small JS snippet dropped into the page (same mechanical pattern as a GA tracking script). It buffers events locally, batches them, and ships them to our ingestion service — which is exactly the “client SDK → batch → ingestion service → storage” pipeline described earlier for the whole analytics system. The rrweb events themselves would land in our object storage (S3) as compressed blobs, with just the session metadata (session ID, site ID, timestamp, duration) indexed in ClickHouse so we can search/filter sessions before pulling the actual replay data.

## Why AWS Fargate, RDS, ElastiCache and S3?

**Fargate (over EC2 or EKS)** The actual choice here is container orchestration: run-your-own-servers vs. managed. There are three real options: raw EC2 (we manage everything), EKS (Kubernetes, AWS-managed control plane but we still manage nodes/config), or Fargate (serverless containers, no infrastructure to manage at all).

Why Fargate over EC2: With EC2, we'd be patching OS, managing instance sizing, handling scaling groups ourselves, and paying for capacity even when idle. Fargate takes a container image and just runs it — we specify CPU/memory, AWS handles the underlying host entirely.

For a small team, the ops time saved not babysitting instances is worth more than the modest per-vCPU cost premium Fargate carries over raw EC2.

Why Fargate over EKS: Kubernetes is genuinely powerful, but it's also a second full-time skill to run well — networking (CNI), ingress controllers, node pool management, cluster upgrades. For a multi-service app (API, web, ingestion, crawler workers) that isn't yet at a scale where we need Kubernetes-specific features (custom operators, complex multi-tenant scheduling), Fargate gives us “run these containers, scale them, don't think about servers” without taking on a whole second discipline. The honest tradeoff: if this genuinely grows into a large multi-team platform later, EKS gives more control and a bigger ecosystem (Helm charts, operators) — that's a real reason companies eventually migrate to it. But that's a problem to have later, not a day-one requirement, and premature Kubernetes adoption is a classic way to spend more time on infra than product.

**RDS (over self-managed Postgres on EC2)** The choice: run Postgres ourselves on an EC2 box, or let AWS manage it.

Why RDS: Automated backups and point-in-time recovery (restore to any minute in the last N days, not just “whenever we last remembered to pg\_dump”), automated failover with Multi-AZ (a standby replica takes over automatically if the primary dies, versus getting paged and manually promoting a replica), automated minor version patching, and built-in monitoring (Performance Insights shows slow queries without standing up our own monitoring stack). For a small team, the actual risk isn't “will Postgres run fine” — it's “what happens the day the disk fills up, or the instance dies, at 3am with nobody watching it.” RDS absorbs almost all of that operational risk for a cost premium over raw EC2 that's genuinely worth it when there's limited on-call coverage.

The honest tradeoff: RDS costs more than an equivalently-sized EC2 instance running Postgres ourselves, and we lose some low-level tuning flexibility (can't SSH into the box, limited extension support depending on version). Pre-revenue, this is still almost always the right call — the time saved not doing our own backup/failover engineering is worth far more than the dollar difference at small scale.

**ElastiCache (over self-managed Redis on EC2)** Same logic as RDS, applied to Redis. Self-managing Redis means handling persistence config (RDB/AOF snapshotting), failover (Redis Sentinel setup for HA), memory eviction tuning, and patching ourselves. ElastiCache gives us automated backups, Multi-AZ with automatic failover, and CloudWatch metrics out of the box.

Why it matters more than it might seem: Redis is on the critical path for rate-limiting, sessions, and our job queue (BullMQ). If Redis goes down ungracefully and loses data, we're not just losing a cache — BullMQ jobs in flight, active sessions, and rate-limit state all take a hit simultaneously. Paying for managed failover here is cheap insurance against a single point of failure that touches multiple subsystems at once.

The honest tradeoff: for a very small workload, a single t3.micro EC2 running Redis ourselves is meaningfully cheaper, and Redis itself is genuinely simple to run compared to a full RDBMS. If cost is tight early on, self-managed Redis on EC2 is a more defensible corner to cut than self-managed Postgres — Redis has fewer moving parts to get wrong.

**S3 (over EBS/EFS or self-hosted storage)** This one's the most clear-cut of the four. S3 isn't competing with “run it yourself on EC2” the way RDS/ElastiCache are — it's solving a problem EC2 storage (EBS) fundamentally can't:

- EBS volumes are attached to one instance and sized/provisioned upfront. Session recordings, PDF exports, and generated reports don't belong tied to a specific compute in-

stance's disk — they need to be accessible from any service (API, workers, web) regardless of which container happens to be running.

- S3 is effectively infinite, pay-per-GB, no capacity planning. We don't provision "how much storage will we need" — we just write objects and pay for what's stored.
- Built-in lifecycle rules — this matters specifically for session recordings: we can set a rule like "delete recordings older than 90 days" automatically, which is essential once we're storing rrweb blobs for every session across every monitored site — that volume grows fast and needs automatic expiry, not manual cleanup.
- Direct browser upload/download via presigned URLs — useful if we ever want the browser to upload something (or download a large PDF) directly to/from S3 without routing the bytes through our API server.

There isn't really a "self-managed" alternative that makes sense here — running our own object storage (MinIO on EC2) is something we'd only do to avoid AWS lock-in specifically, not for cost or capability reasons. Given we're already all-in on AWS for everything else, S3 is the uncontroversial pick.